

Web Features Technical Specification

This document provides an A-Z detailed technical specification of all the Web Portal Features, sequentially numbered from W01 to W43. Features are categorized strictly by their logical domains to guide the Web Frontend Development team.

1. Dashboard & Core Metrics

W01: Welcome Message & Date

- **Data Source:** `UserModel`
- **Implementation:** The web portal fetches the currently logged-in user's `name` from the `UserModel` and uses the browser's time to determine the greeting (e.g., "Good Morning, {name}").

W02: Member & Task Statistics

- **Data Source:** `UserModel` , `LeaveApplicationModel` , `TaskModel`
- **Implementation:**
 - *Total Active/Alumni:* Query `UserModel` where `status` is 'active' or 'alumni'.
 - *Members on Leave:* Query `LeaveApplicationModel` where `status` = 'approved' and the current date falls within the `duration` object.
 - *Active Tasks:* Query `TaskModel` for tasks assigned to the user where `status` is 'in_progress'.

W03: Today's Active Meeting

- **Data Source:** `MeetingModel`
- **Implementation:** Query `MeetingModel` where `date_time` matches today's date. The widget displays `title` , `date_time` , and `agenda` . Clicking it routes to the Meeting Details page, passing the `_id` .

W04: My Tasks

- **Data Source:** `TaskModel`
- **Implementation:** Query `TaskModel` where the user's `_id` is inside the `assigned_to` array. Displays `name` and domain (via `domain_id`).

W05: Quick Action Buttons

- **Implementation:** UI-level shortcuts (e.g., "Apply Leave", "New Task") that navigate to respective web pages. Visibility controlled by `RoleModel` policies.

W06: Community Updates & Task Load Graph

- **Data Source:** `UpdatesModel` , `TaskModel`
- **Implementation:**
 - *Updates:* Query `UpdatesModel` for the latest broadcasts (using `title` and `content`).
 - *Task Load:* Aggregation query on `TaskModel` grouping by `status` and `domain_id` to render a visual chart (e.g., using Chart.js/Recharts) on the dashboard.

W07: Global Search Bar (Universal Search)

- **Data Source:** `UserModel` , `TaskModel` , `MeetingModel` , `RepositoryModel`
- **Implementation:** A universal search bar located in the top navigation bar of the web portal. It executes text searches across tasks, meetings, members, and documents for quick routing.

2. Tasks & Calendar

W08: Calendar View

- **Data Source:** `TaskModel` , `MeetingModel`
- **Implementation:** Fetches all tasks and meetings for the month. Uses `deadline` (Tasks) and `date_time` (Meetings) to visually pin them on a full-page web calendar grid.

W09: Tasks List View

- **Data Source:** `TaskModel` , `NotificationModel`
- **Implementation:** Lists tasks sorted by `deadline` in a web data table. Includes a reassignment feature which triggers a new `NotificationModel` document for assigned/unassigned members.

W10: Add Task Button (Admin/Authority)

- **Data Source:** `TaskModel` , `RoleModel` , `NotificationModel`
- **Implementation:** Checks IAM RBAC. Opens a modal or page to create a `TaskModel` document with `name` , `deadline` , `mentor_ids` , `description` , `type` , `priority` , `attachments` , `assigned_to` , and `domain_id` .

W11: Task Filters (Completed, In Progress, Not Started)

- **Data Source:** `TaskModel`
- **Implementation:** UI dropdowns/tabs that query the tasks list strictly by the `status` field in `TaskModel` .

W12: Task Workflow & Table

- **Data Source:** `TaskModel`
- **Implementation:** Users accept tasks (updates `status` to 'in_progress'). Submission appends an object to `TaskModel.submissions` . Mentors can review via `TaskModel.reviews` array.

W13: Task Activity Log (Audit Trail)

- **Data Source:** `TaskModel`
- **Implementation:** A timeline component in the Task Details page rendering the `TaskModel.history` array to track assignments and status changes.

3. Meetings

W14: Meeting Dashboards

- **Data Source:** `MeetingModel`
- **Implementation:** Fetches and displays all meetings as informational cards in a responsive web grid.

W15: Create Meeting

- **Data Source:** `MeetingModel` , `RoleModel`
- **Implementation:** Validates admin authority via RBAC. Creates a `MeetingModel` document populated with `title` , `agenda` , `venue` , `target_batch` , and `target_domain` .

W16: Meeting Filters

- **Data Source:** `MeetingModel`
- **Implementation:** Web UI filters to categorize meetings by `status` ('upcoming', 'ongoing', 'completed').

W17: Meeting Summary & Attendance

- **Data Source:** `MeetingModel` , `AttendanceModel`
- **Implementation:** Admins update `MeetingModel.mom_content` using a rich text editor. Attendance tracking creates/updates `AttendanceModel` documents linking `meeting_id` and `user_id` .

W18: Integrated Meeting

- **Implementation:** Uses `MeetingModel.venue.link` to open Google Meet or other platforms in a new browser tab.

4. Registry / Analytics

W19: Tables of Works

- **Data Source:** `TaskModel`
- **Implementation:** Queries `TaskModel` where `status` = 'completed'. Displays who completed it. Export feature parses this JSON to trigger a CSV file download

for the admin.

W20: Tables of Meeting

- **Data Source:** `AttendanceModel` , `MeetingModel`
- **Implementation:** Aggregates `AttendanceModel` for a specific `meeting_id` to count present vs. absent members.

5. Leave Application

W21: Leave Application Management

- **Data Source:** `LeaveApplicationModel`
- **Implementation:** User fills a web form to create a record with `user_id` , `leave_type` , `target` , `duration` , and `reason` . Defaults to `status` = 'pending'.

6. Repository

W22: Essential Documents

- **Data Source:** `RepositoryModel`
- **Implementation:** Queries `RepositoryModel` based on user's domain and role. Displays `name` , `type` , and provides hyperlink buttons from `file_urls` .

7. Forms

W23: Active Form Cards

- **Data Source:** `FormModel`
- **Implementation:** Queries `FormModel` where `deadline` > current time.

W24: Form Fillup Fields

- **Data Source:** `FormModel` , `FormSubmissionModel`

- **Implementation:** Dynamically renders web form inputs based on `fields` array in `FormModel`. On submit, creates a `FormSubmissionModel` linking `form_id` and `user_id`.

W25: Form Instructions

- **Data Source:** `FormModel`
- **Implementation:** Displays instructions dynamically read from the `FormModel`.

W26: Form Responses Dashboard (Admin Feature)

- **Data Source:** `FormSubmissionModel`, `FormModel`
- **Implementation:** Admin can view all submissions for a specific form. Renders the JSON `responses` in a detailed web table or analytical chart, with an option to export to CSV.

8. Profile, Settings & Auth

W27: User Profile

- **Data Source:** `UserModel`, `TaskModel`
- **Implementation:** Fetches `profile_pic`, `name`, `domain`, and `task_count` from models.

W28: Sidebar Options

- **Implementation:** Global Web UI navigation menu (Left Sidebar for Desktop, Hamburger menu for Mobile browsers).

W29: Dark / Light Mode

- **Implementation:** Web UI theme toggle, saves preference in LocalStorage or cookies.

W30: Developer Mode

- **Implementation:** Hidden toggle that enables extra debugging tools within the

web console or UI.

W31: Login & Auth

- **Data Source:** `UserModel`
- **Implementation:** Authenticates via email / password or Google OAuth. Saves JWT token in HTTP-only cookies or LocalStorage.

W32: Detailed Documentation

- **Implementation:** Static web pages serving as a guide/help section for normal members.

W33: Terms and Conditions

- **Implementation:** Static web page displaying the official rules.

W34: Web Notification Center (Bell Icon)

- **Data Source:** `NotificationModel`
- **Implementation:** A bell icon in the navbar opening a dropdown list of the user's notifications.

W35: Team Directory (For Normal Members)

- **Data Source:** `UserModel`
- **Implementation:** A directory/search page allowing normal members to view their peers.

W36: Forgot Password & Reset Flow

- **Data Source:** `UserModel`
- **Implementation:** Standard OTP/Email flow for password reset on the web portal.

9. Admin Authority

W37: Admin Dashboard

- **Data Source:** Aggregation across Models
- **Implementation:** Accessible only if `RoleModel` grants admin policies. Shows high-level metrics and control panels.

W38: Member Management (List of all members)

- **Data Source:** `UserModel` , `RoleModel`
- **Implementation:** Admins can modify `status` (make alumni) and update `volatile_data` in the `UserModel` via a management table.

W39: Application Approvals

- **Data Source:** `LeaveApplicationModel`
- **Implementation:** Admins fetch 'pending' leaves. On action, updates `status` and `reviewed_by` .

W40: Dynamic Roles & Permissions Manager (IAM UI)

- **Data Source:** `RoleModel` , `PolicyModel`
- **Implementation:** Dedicated Superadmin page in the Web Portal to create custom roles, assign specific access policies, and link them to users dynamically.

W41: Bulk Member Upload (CSV)

- **Data Source:** `UserModel`
- **Implementation:** Allows Admins to upload a CSV file of member details via a drag-and-drop web zone for batch insertion.

10. Chat Feature

W42: Core Chat

- **Data Source:** `ChannelModel` , `ChatModel`

- **Implementation:** Group chat using `ChannelModel` and `ChatModel` with WebSockets running in the browser.

W43: Advanced Chat (File Sharing & Mentions)

- **Data Source:** `ChatModel` , `RepositoryModel` , `NotificationModel`
- **Implementation:** Enhances the web chat by allowing file attachments (drag-and-drop) and `@mentions` .